

JavaScript Code

```
1  require('dotenv').config();
2
3  const http = require('http');
4  const {URL} = require('url');
5  const {ObjectId} = require('mongodb');
6  const { connectDB, getDB, closeDB } = require('./db.js');
7  const {
8    sendJson,
9    readJson,
10   parseCookies,
11   randomToken,
12   hashToken,
13   hashPassword,
14   verifyPassword
15 } = require('./utils.js');
16
17 const {
18   createSession,
19   sessionCookie,
20   clearSessionCookie,
21   getAuthenticatedUser,
22   requireRole
23 } = require('./auth.js');
24
25 const PORT = Number(process.env.PORT || 3000);
26
27 const allowedOrigins = new Set(
28   String(process.env.ALLOWED_ORIGINS || '').
29   split(',').
30   map(v=>v.trim()).
31   filter(Boolean)
32 );
33
34 function applyCors(req,res){
35   const origin = req.headers.origin;
36
37   if(origin && allowedOrigins.has(origin)){
38     res.setHeader('Access-Control-Allow-Origin',origin);
39     res.setHeader('Access-Control-Allow-Credentials',true);
40     res.setHeader('Access-Control-Allow-Headers','Content-Type');
41     res.setHeader('Access-Control-Allow-Methods','GET,POST,OPTIONS');
42     res.setHeader('Vary','Origin');
43   }
44 }
45
46 function isValidObjectId(value){
47   return ObjectId.isValid(value) && String(new ObjectId(value))===value;
48 }
49
50 async function login(req,res){
```

```
51     const {email,password} = await readJson(req);
52     if(!email || !password){
53         return sendJson(res,400,{message:'email and password required'});
54     }
55
56     const db = getDB();
57     const user = await db.collection('users').findOne({email:email.toLowerCase()});
58     if(!user || !verifyPassword(password,user.passwordHash)){
59         return sendJson(res,401,{error:'Invalid Credentials'});
60     }
61
62     const {rawToken,expiresAt} = await createSession(user);
63     return sendJson(
64         res,200,
65         {message:'Login successful',user: { id: user._id, name: user.name, role:
user.role } },
66         {'Set-Cookie':sessionCookie(rawToken,expiresAt)}
67     );
68 }
69
70 async function logout(req,res,user){
71     if(user?.sessionTokenHash){
72         await
73         getDB().collection('sessions').deleteOne({tokenHash:user.sessionTokenHash});
74     }
75     return sendJson(res,200,{message:'Logged out'},{'Set-
Cookie':clearSessionCookie()});
76 }
77
78 async function route(req,res){
79     applyCors(req,res);
80     if(req.method==='OPTIONS'){
81         const origin = req.headers.origin;
82         if(origin && !allowedOrigins.has(origin)){
83             return sendJson(res,403,{error:'Origin not allowed by CORS policy'});
84         }
85         res.writeHead(204);
86         return res.end();
87     }
88     const url = new URL(req.url,`http://${req.headers.host || 'localhost'}`);
89
90     if(req.method==='GET' && url.pathname=== '/health'){
91         return sendJson(res,200,{ok:true,service:'Healthcare backend system'});
92     }
93
94     if(req.method==='POST' && url.pathname=== '/auth/login'){
95         return login(req,res);
96     }
97
98     const user = await getAuthenticatedUser(req);
99     if(!user) return sendJson(res,401,{error:'Authentication required'});
100
101     if(req.method==='GET' && url.pathname=== '/auth/me'){
```

```
102         return sendJson(res, 200, {name: user.name});
103     }
104     if(req.method === 'POST' && url.pathname === '/auth/logout'){
105         return logout(req, res, user);
106     }
107
108     return sendJson(res, 400, {error: 'route not found'});
109 }
110
111 async function start(){
112     await connectDB();
113     const server = http.createServer((req, res) => {
114         route(req, res).catch(err => {
115             console.log(err);
116             if(!res.headersSent){
117                 sendJson(res, 500, {message: 'Internal server error'});
118             } else {
119                 res.end();
120             }
121         });
122     });
123     server.listen(PORT, () => {
124         console.log(`Healthcare backend is running on http://localhost:${PORT}`);
125     });
126
127 }
128
129 start().catch(err => {
130     console.log('Startup failed', err);
131     process.exit(1);
132 });
```